

Kimi-K2 后端下三款编码 Agent 的忠实复评： SSE 解析错误修复后的画像翻转

关嘉伟*

OpenClaw Research

2026 年 4 月

Abstract

我们在 Kimi-K2 统一后端下复评了三款开源编码 Agent(openclaw、hermes、claude-code) 的 token 行为。起因很无趣：之前 mem5_full_v2 报告所用的 SSE 解析器对 data: 行的空格太敏感，Kimi 有时不带空格，结果 claude-code 的 pure_input 被解析成近零、cache 命中率冲到 99%，让人误以为它和 hermes 是两种完全不同的 profile。修掉解析器、再在 SWE-bench Verified 的三道非饱和任务（astropy-12907、django-11138、django-13212）上重跑以后，那种反差就没了：两个走 Anthropic 协议的 Agent 在 cache_read 占比（97.1% vs 97.2%）、pure_input（80k vs 84k）和工具调用数上几乎是同一张画像。openclaw 只完成了 2/6，其余 4 次要么死循环打满 30 分钟、要么发了一次工具调用就提前退出。顺带也修了一个 adapter 会话元数据回退 bug——它会让静默退出那轮“继承”上一轮的累计 token——并对受影响的 6 条历史记录做了标注。真正拉开 Agent 差距的不是 cache 策略，是 agent-loop 本身的稳定性。

关键词：编码 Agent、SWE-bench、Kimi、Cache 计价、SSE 流解析、忠实遥测

1 引言

在一个统一的 LLM 后端下横向对比多款 Agent，最容易踩的坑是”对比的不是 Agent 本身，而是数据采集管线的 bug”。我们此前发表的 mem5_full_v2 报告中给出过一个看起来很干净的三分形态：openclaw 几乎不使用 cache，hermes 的 cache 约占输入的 95%，而 claude-code 的 pure_input”塌到零”、cache 占比逼近 100%。这个反差当时被我们归因为三款 Agent 的上下文复用策略差异。

事后我们在一次 smoke test 里发现，claude-code 所依赖的 OpenAI→Anthropic shim 层在解析 Kimi 的 SSE 流时对 data: 行的空格要求不符合 WHATWG 规范：Kimi 返回的 data: 后面有时不带空格，slice(6) 会把第一个字节（往往是 JSON 的 {）截掉，整个 assistant turn 就会被判无内容。shim 层原本会把这种情况当成”上游啥也没给”，于是返回的 modelUsage 字段里的 inputTokens 只剩 cacheReadInputTokens，pure_input（按 input - cache_read - cache_write 计算）自然就是 0。

所谓”三分天下”其实是解析器 bug 的影子。修复该 bug（Kimi 的 data: 无空格路径改走 slice(5).trimStart()）之后，我们在比 astropy-12907 更难的两道 django 任务上重跑了 3 款 Agent × 5 轮 × 3 任务。这篇报告就是那次复跑的结果。

*通讯: j.guan.sense@gmail.com

这次复跑主要做了三件事。首先是把原报告里”claude-code 独立形态”的结论明确撤回——SSE 修复以后，claude-code 和 hermes 的 cache 画像都落在 97.1% 量级上，肉眼基本看不出区别。然后是把 benchmark 从饱和的 astropy-12907 (三家都 100%) 换到两道 django hard，三家通过率这才第一次分开 (6/6 vs 6/6 vs 2/6)。最后顺手修了一个数据完整性 bug: openclaw 的 adapter 在当前轮没新会话事件时，会把 after_session["meta"] 的累计计数当成本轮的 token 用量；修复已经回填到历史数据。

2 背景与实验配置

2.1 统一后端

三款 Agent 都通过其 CLI 路由到 Kimi 的 kimi-for-coding 模型: openclaw 原生支持 Kimi provider, hermes 走 custom:kimi 配置, claude-code 经由 openclaude 的 OpenAI→Anthropic shim。我们不修改 Agent 的系统提示或工具集，只统一后端。

2.2 数据集选择

我们从 SWE-bench Verified 里选了三道实例：

- astropy__astropy-12907: 原 mem5_full_v2 的 pilot 题，三家都 100% 通过，用作”饱和参考”。
- django__django-11138: difficulty = 15 min - 1 hour，用作第一道区分题。
- django__django-13212: difficulty = 1-4 hours，用作第二道区分题。

两道 django 难题按 SWE-bench 的人类难度标注筛选。这是我们这次从饱和 benchmark 逃出来的关键选择。

2.3 Memory-enabled 多轮协议

沿用 mem5_full_v2 的 memory-enabled profile: 第 1 轮前清空 runtime state，第 2-5 轮复用前一轮的 state 目录。每个 Agent × 每道题 = 5 轮，总计 $3 \times 3 \times 5 = 45$ 次调用。

2.4 Quota 截断与”完整窗口”

本次实验的 Kimi 配额在 r03 之后逐步耗尽，表现为: claude-code 从 r03 开始 HTTP 403, hermes 从 r04 开始失败。能够对三家做无偏对比的完整窗口只有 r01+r02，也即 $3 \text{ 任务} \times 2 \text{ 轮} \times 3 \text{ Agent} = 18$ 次有效调用。后文所有”干净对比”的结论都仅使用这 18 次数据。

3 数据完整性修复

3.1 SSE 解析 bug (openclaude)

openclaude 的 OpenAI shim 里负责把 OpenAI-style SSE 转译成 Anthropic-style content blocks 的代码，按 data:_ 六字节前缀切。WHATWG SSE 规范允许 data: 后面不带空格。Kimi 的 /chat/completions 流实际上大多数时候会加空格，但在某些模型(包括 kimi-for-coding)和某些 chunk 边界上会省略。当第一个非空 data 行没有空格时，slice(6) 会吃掉首个 JSON 字符，chunk 被当成解析失败丢弃。

修复落在 openclaude 的 src/services/api/openaiShim.ts: 探测到 data: 前缀后,用 slice(5).trimStart() 而不是硬 slice(6)。修复后在 Kimi coding 端点上 cache_read 恢复到 3.4M 量级 (之前的 bug 下 cache_read 仍然被记录, 因为它来自 modelUsage 汇总字段, 不走 SSE 解析路径)。

3.2 静默退出继承累计 token (openclaw)

我们的 adapter 用三层回退从 openclaw 的 CLI 读取每次调用的 token 用量: (1) 本轮追加的 session 事件里的 assistant usage、(2) 本次 CLI 输出的 payload.agentMeta.lastCallUsage、(3) after_session["meta"] 里的 inputTokens/outputTokens。第 (3) 层本意是对“CLI 输出了但 session 文件没写完”的边界情况兜底, 但它实际是会话终身累计计数。

当某一轮 CLI 静默秒退 (下文 §4.4 会讨论的 openclaw 失败模式之一), 前两层都是零, 第三层就把上一轮的累计数字写进当前轮的记录。表现为 r04 和 r05 的 tokens_in 会“刚好等于”r03 的真实值, 哪怕 r04 只跑了 15 秒、工具调用数是 0。

我们加了一道 has_new_activity 门——只要本轮没观察到任何新活动, 就不再回退到累计 meta, 而是以零值记录并把 error 字段置为 silent_exit_no_api_activity。同时对已生成的 6 条被污染记录做了回填: 零化当前计数, 把原值存到 usage_details.inherited_cumulative, 标明 backfilled。

这个修复不是 Agent 本身的行为更改, 只是让遥测更忠实。openclaw 自己“秒退”这件事是它当前 memory-enabled 模式下的真实表现, 我们不去掩盖它。

4 结果

4.1 Cache 画像收敛

表 1 是 r01+r02 所有成功 run 的平均画像。hermes 和 claude-code 在 cache_read 占比、pure_input、工具调用数这三个关键指标上几乎完全对齐, 两者的差异在 token 量级上小于单轮波动。

Table 1: r01+r02 成功 run 的 token 与工具画像 (平均值)。pure_input 单位 token, cache_read 和 total 单位 M-token。

Agent	n	pure_input	cache_read	total	cache%	tools
hermes	6	80,196	3.23 M	3.33 M	97.1%	66
claude-code	6	84,259	3.76 M	3.87 M	97.2%	75
openclaw	2	141,632	2.06 M	2.22 M	92.7%	37

与原 mem5_full_v2 报告相比: 原文给出的 claude-code“pure_input=0/cache%=99.1”彻底消失, 取而代之的是与 hermes 几乎平行的 97.2%/84k。解析 bug 修复后这个结果是稳定重现的 (我们在独立的 smoke test 上验证过, 见 §1)。

观察 1. 在同一 LLM 后端 (Kimi)、同一 Anthropic-messages API 协议下, hermes 与 claude-code 的 cache 利用策略本质来自上游 SDK 的缓存协商行为, 而不是 Agent 框架的独立设计选择。

4.2 任务通过率: 一次有意义的分化

表 2 是 r01+r02 窗口内三家的任务通过情况。

Table 2: r01+r02 任务通过率矩阵。括号内为失败模式简述。

Agent	astropy-12907	django-11138	django-13212
hermes	2/2	2/2	2/2
claude-code	2/2	2/2	2/2
openclaw	0/2 (timeout, 30 min)	1/2	1/2

hermes 与 claude-code 两家都在 6/6 上顶格。openclaw 2/6 = 33%。原 mem5_full_v2 的 benchmark (只含 astropy-12907) 在三家都 100% 上饱和, 无法暴露这种差距; 换上两道 django hard 之后, 三款 Agent 的能力差才第一次被采样到。

4.3 效率: hermes 略胜

限制在 6/6 都通过的两个 Agent 之间看效率: hermes 的平均延迟 470s 比 claude-code 的 596s 低 21%, 总 token 消耗 3.33M 比 3.87M 低 14%, 工具调用 66 比 75 少 12%。差距不大但方向一致, 在我们这 6 个成功运行上都成立。

4.4 openclaw 的两种失败模式

openclaw 的 4 次失败不是同一种:

死循环型。astropy-12907 的 r01/r02 均打满 30 分钟 timeout, 分别做了 433 和 448 次工具调用。人工抽查 session 文件可见 Agent 在反复 read/grep 同一组文件而不做 write。

过早放弃型。django-13212 的 r02 只发了 1 次工具调用, 554 秒空转后无 patch 退出。session 事件里第一次工具调用之后 Agent 立刻发了一个空 assistant turn 结束会话。

这两种模式都不是 token 预算问题——前者超时而非超 token, 后者主动结束得太早。我们倾向把这归为 agent loop 控制逻辑对难题的鲁棒性不够, 而不是 LLM 后端差异。

5 讨论

5.1 对原报告结论的回撤

原报告基于“三分形态”提出的几个衍生结论——比如“claude-code 通过激进 cache 获得了最低的 cost_tokens”——在这次修复后的数据下不成立。三款 Agent 里 openclaw 的 cache% 仍然略低 (92.7%), 但差距只有 4-5 个百分点, 且样本量 (n=2 成功 run) 不足以支撑窄置信区间。我们撤回原报告第 3、4 节的排序结论。

5.2 忠实遥测作为方法论要求

两个 bug——上游 shim 的 SSE 解析、我们自己 adapter 的累计 meta 回退——都不会让脚本崩溃, 只是悄悄污染数字。它们能被注意到纯粹是因为数字“看起来太整齐”了: pure_input=0, r04/r05 的 token 数刚好等于 r03。靠这种巧合发现 bug 并不牢靠。与其依赖运气, 不如让遥测本身提供检查点: 每轮记一次 final_call_input_tokens (这次调用的 usage, 不是累计); 加一个显式的 has_new_activity 布尔字段把“本轮真干活”和“本轮复用了之前的状态但没产生新活动”分开; agent 异常退出 (静默、panic、超时) 写明确的错误码, 而不是默认写入 error=null。这三样不用多花什么成本, 但能省掉下一次“数字太整齐”才回头查的时间。

5.3 局限

这次实验的主要局限是样本量。受 Kimi 配额约束，可用来横向对比的完整窗口只有 r01+r02（每 Agent 6 次成功调用）。结论在这 6 次上稳定，但我们没有 bootstrap 置信区间。下一步我们会在配额恢复后扩到完整 5 轮并加 instance 数到 15-20 道。

6 结论

数据采集管线自己有 bug 的时候，Agent 之间看起来的”显著差异”常常只是管线的差异。把解析 bug 和遥测 bug 都修掉之后，两款走 Anthropic 协议的 Agent 在 Kimi 后端上的 cache 画像几乎重合；真正把三款 Agent 分开的，是 **openclaw** 在硬任务上 agent-loop 的稳定性——33% 的通过率，加上两种可重现的失败模式（死循环和过早放弃）。我们从这次经验里抽出的三点想法，更像是下一轮基准实验的自我提醒：挑几道难度没饱和的任务做起点，通过率之外顺便把失败模式分布也报出来；发表之前最好对遥测链路做一次”数字看起来是不是太整齐了”的复盘。

致谢

感谢 Moonshot 提供 Kimi kimi-for-coding 端点。感谢 openclaude、hermes、openclaw 三个项目的维护者。本实验由 GB10 集群完成。

References

- [1] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.
- [2] 关嘉伟. *mem5_full_v2: Memory-Enabled Runtime Profile* 下三款 Agent 的 Token 行为对比. Open-Claw Research Technical Report, 2026 年 4 月.
- [3] WHATWG. *HTML Living Standard §9.2 Server-sent events*. <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [4] Moonshot AI. *Kimi K2 API Reference —kimi-for-coding model*. <https://platform.moonshot.cn/>